

Cara Menjalankan Program — Awal sampai Akhir

How to Run — Start to Finish

Tugas Besar Big Data · ETL & ELT + Dashboard

ID: Panduan ini sudah memuat **semua perbaikan nyata** yang ditemukan saat menjalankan pipeline (image, izin, memori, dll). Ikuti urutannya.

EN: This guide already includes **all the real fixes** discovered while running the pipeline. Follow the order.

0. Prasyarat / Prerequisites

- **Docker Desktop** (WSL2 backend, Windows 11) — aktif/running.
- **FRED API key** gratis → isi di `.env`.
- **Tableau Desktop + Power BI Desktop + Hive ODBC driver 64-bit** (untuk dashboard).
- Disk lega di **C:** (≥15 GB) — image Spark+Hive+torch besar.
- RAM: stack penuh ~6–7 GB di VM Docker (lihat catatan memori §6).

```
# siapkan .env Copy-Item .env.example .env # lalu isi FRED_API_KEY=...
```

1. Build image / Build images

ID: Membangun image Spark (+deps Python) & Hive (+driver Postgres). Pertama kali agak lama (torch ~beberapa menit).

EN: Builds the Spark (+Python deps) and Hive (+Postgres driver) images. First time is slow.

```
docker compose build
```

Perbaikan sudah diterapkan / fixes already applied: base image bitnamilegacy/spark:3.5 (bitnami lama dihapus 2025); pyspark tidak di-reinstall; entri `/etc/passwd` uid 1001; driver Postgres di-COPY (bukan ADD URL yang menghasilkan 0 byte).

2. Nyalakan stack / Start the stack

```
docker compose up -d docker compose ps # semua harus Up; postgres & kafka "healthy"
```

Verifikasi metastore / verify metastore (init schema sukses):

```
docker compose logs hive-metastore --tail 5 # cari "Initialized schema successfully"
```

Port: Spark UI :8080 · HiveServer2 JDBC :10000 · HiveServer2 UI :10002 · Kafka :9092 · Kafka-UI :8085.

3. (Sekali) CTGAN → sumber 1 / (Once) CTGAN → source 1

ID: CTGAN berat (torch) dan bisa meng-OOM Hive bila dijalankan bersamaan. Jalankan **sekali** dengan Hive dimatikan sementara, lalu **salin** hasilnya ke `raw/` sebagai sumber 1. Run berikutnya **tidak** perlu CTGAN.

EN: CTGAN is heavy and can OOM Hive. Run it **once** with Hive stopped, then **copy** its output to `raw/` as source 1. Later runs **skip** CTGAN.

```
docker compose stop hive-metastore hiveserver2 docker compose exec spark-master python
/app/synthetic/train_ctgan.py --n 150000 --epochs 10 --seed 42 Copy-Item
synthetic/synthetic_credit_clients.csv raw/ # -> raw/synthetic_credit_clients.csv (SUMBER 1) docker compose
start hive-metastore hiveserver2
```

Sudah jalan / already done: 150,000 nasabah, balance {non-default 0.7788, default 0.2212}.
Jika regenerasi tak diperlukan, **lewati langkah ini** (file sudah ada di `raw/`).

4. Extract FRED (sumber 2) / Extract FRED (source 2)

```
docker compose exec spark-master python /app/etl_pipeline/extract.py
```

ID: Mengisi `raw/fred_*_2005.json` (4 seri × 12 obs). Jika ada seri gagal "Network is unreachable" (flaky), **ulangi** perintahnya (idempoten).

EN: Writes the FRED JSON files; if a series fails with a transient network error, just re-run.

5. Jalankan pipeline / Run the pipeline

PENTING / IMPORTANT (ID): tahap yang **menulis ke warehouse Hive** dijalankan dengan **-u root**. Sebabnya: volume warehouse dipakai dua uid berbeda (Hive=1000, Spark=1001); root menembus beda kepemilikan, file tetap world-readable untuk HiveServer2/ODBC.

(EN): stages that **write to the Hive warehouse** run with **-u root** to bypass the uid-1000/1001 ownership clash on the shared volume (files stay world-readable for ODBC).

```
# 5a. ETL transform (PySpark) -> staging parquet (baca raw/synthetic_credit_clients.csv) docker compose exec
spark-master spark-submit /app/etl_pipeline/transform.py # 5b. ETL load -> star schema bigdata_etl + 8 query
analitik docker compose exec -u root spark-master spark-submit /app/etl_pipeline/load.py # 5c. ELT
extract+load -> bigdata_elt.raw_* ; lalu transform in-warehouse (window/OLAP) docker compose exec -u root
spark-master spark-submit /app/elt_pipeline/extract_load.py docker compose exec -u root spark-master
spark-submit /app/elt_pipeline/run_transform.py # 5d. KPI dashboard -> kpi_* di bigdata_etl & bigdata_elt
docker compose exec -u root spark-master spark-submit /app/dashboard/build_dashboard_kpis.py # 5e. (opsional)
perbandingan ETL vs ELT + diagram arsitektur docker compose exec spark-master python
/app/analysis/compare_pipelines.py python architecture_diagram.py
```

Hasil yang diharapkan / expected (sudah terbukti / proven): transform 900,000 baris fakta & 6/6 validasi PASS; load 4 tabel star schema; ELT `elt_*` (incl. 900k `elt_fact_monthly`); KPI di kedua DB.

Jika load/ELT gagal `LOCATION_ALREADY_EXISTS` atau `Mkdirs failed` (sisa run lama):
`powershell docker compose exec -u root spark-master sh -c "chmod -R 777 /opt/hive/data/warehouse; rm -rf
/opt/hive/data/warehouse/bigdata_etl.db /opt/hive/data/warehouse/bigdata_elt.db"`
lalu ulangi. (Skrup kini `DROP DATABASE ... CASCADE` otomatis → idempoten.)

6. Catatan Memori / Memory Note (penting!)

ID: VM Docker ~7.5 GB. Stack penuh + CTGAN bisa OOM (Hive ter-kill, exit 137). Solusi:
- Jalankan CTGAN **terpisah** (Hive dimatikan) — sudah di §3.

- Sebelum konek **Power BI/Tableau**, matikan layanan yang tak perlu agar HiveServer2 lega:

```
docker compose stop spark-worker kafka kafka-ui # sisakan postgres + hive-metastore + hiveserver2
docker compose up -d --force-recreate hiveserver2
```

EN: the Docker VM is ~7.5 GB; the full stack + CTGAN can OOM Hive. Run CTGAN separately, and stop Spark/Kafka before connecting BI tools so HiveServer2 has headroom.

7. Verifikasi Hive / Verify Hive (beeline)

```
docker compose exec hiveserver2 beeline -u "jdbc:hive2://localhost:10000" -e "SHOW TABLES IN bigdata_elt;
SELECT * FROM bigdata_elt.kpi_overall;"
```

Hasil terverifikasi / verified: 11 tabel di `bigdata_elt`; `kpi_overall` = default_rate 0.2212, total_clients 150000.

8. Dashboard — Tableau (ETL) & Power BI (ELT)

ID: Pastikan \$6 (HiveServer2 lega) + \$7 (data ada). Lihat detail di `dashboard/HIVE_ODBC_CONNECTION.md` & `dashboard/DASHBOARD_SPEC.md`.

Koneksi ODBC (perbaikan error "Unexpected response received from server"):

- Jangan pakai "Sample Cloudera Hive DSN" mentah — **konfigurasikan dulu** DSN-nya.
- ODBC Data Sources (64-bit) → System DSN → Configure:
- Host localhost, Port 10000, Database `bigdata_elt` (Power BI) / `bigdata_etl` (Tableau)
- Hive Server Type **HiveServer2**, Mechanism **User Name**, User `hive`
- **Thrift Transport = Binary** (jika SASL menghasilkan "Unexpected response"), **SSL = OFF**
- **Test** → SUCCESS.
- Power BI: **Get Data** → **ODBC** → **[DSN]** → **Connect** → centang `kpi_*` → **Load**.

EN: the "Unexpected response" error = sample DSN with wrong transport/SSL. Configure the DSN (Host localhost:10000, DB `bigdata_elt`, HiveServer2, User Name `hive`, Thrift Transport=Binary, SSL off, Test→SUCCESS), then Power BI Get Data → ODBC.

Alternatif tanpa ODBC / no-ODBC fallback: export `kpi_*` ke CSV lalu *Get Data* → *Text/CSV*:

```
docker compose exec hiveserver2 beeline -u "jdbc:hive2://localhost:10000" --outputformat=csv2 -e "SELECT *
FROM bigdata_elt.kpi_default_by_demographic;" > kpi_demographic.csv
```

9. Matikan / Teardown

```
docker compose down # stop + hapus container (volume tetap)
docker compose down -v # + hapus volume (mulai bersih)
```

10. Ringkasan Urutan / Order Summary

1. `.env` → 2. `docker compose build` → 3. `up -d` → 4. (sekali) CTGAN → salin ke `raw/` →
2. extract FRED → 6. transform → 7. load (`-u root`) → 8. ELT `extract_load` + `run_transform` (`-u root`) → 9. dashboard KPIs (`-u root`) → 10. (stop Spark/Kafka) → 11. konek Tableau/Power BI.